

Fine-Tuning LLMs and RLHF

Naeemullah Khan

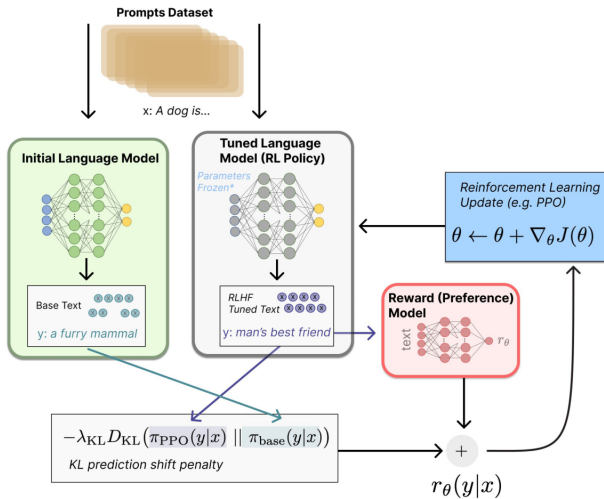
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026



1. Motivation
2. Learning Outcomes
3. Fine-Tuning Methods for LLMs
 - 3.1 Categories of Fine-Tuning
 - 3.2 Full Fine-Tuning (FT)
 - 3.3 Parameter-Efficient Fine-Tuning (PEFT)
4. Supervised Fine-Tuning (SFT)
 - 4.1 What is SFT?
 - 4.2 Datasets for SFT
 - 4.3 Limitations of SFT
5. RLHF — Reinforcement Learning with Human Feedback
 - 5.1 Why RLHF?

2. RLHF Pipeline
3. Reward Model (RM)
4. RLHF Training (with PPO)
5. Direct Preference Optimization (DPO)
6. LoRA and Quantized LoRA
 1. What is LoRA?
 2. Benefits of LoRA
 3. What is QLoRA?
 4. Applications of LoRA / QLoRA
7. Limitations of Fine-Tuning and RLHF
8. Future Directions
9. Summary

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- ▶ **We need:**
 - Adaptability (domain tuning)
 - Alignment (user intention)
 - Efficiency (cost-effective methods)

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- ▶ **We need:**
 - Adaptability (domain tuning)
 - Alignment (user intention)
 - Efficiency (cost-effective methods)
- ▶ Fine-tuning and RLHF bridge the gap between general intelligence and practical usability.

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

After this session, you will be able to:

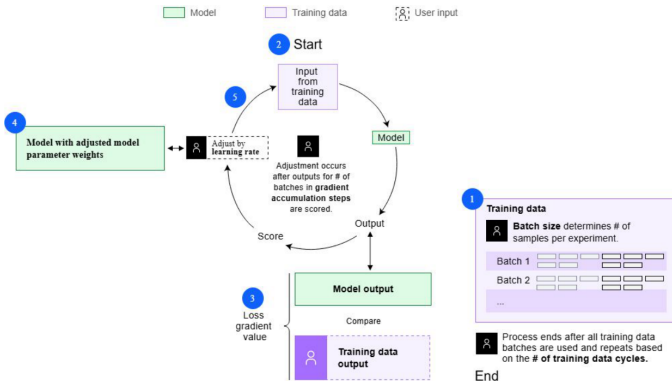
- ▶ Explain and compare different fine-tuning methods for LLMs
- ▶ Understand the pipeline of Supervised Fine-Tuning and RLHF
- ▶ Describe PPO (Proximal Policy Optimization) and DPO (Direct Preference Optimization)
- ▶ Apply Low-Rank Adaptation (LoRA) and Quantized LoRA for efficient tuning
- ▶ Analyze trade-offs, limitations, and future directions of LLM adaptation

Fine-Tuning Methods for LLMs

Method	Description	Params	Efficiency
Full FT	Retrain all weights	All	Expensive
Adapter	Add small bottlenecks (Houlsby)	Few	Efficient
Prefix	Tune soft prompts	Few tokens	Efficient
LoRA/QLoRA	Low-rank weight deltas	Very few	Very Efficient
Instr. Tuning	Tune on instr. datasets	All/Part	-

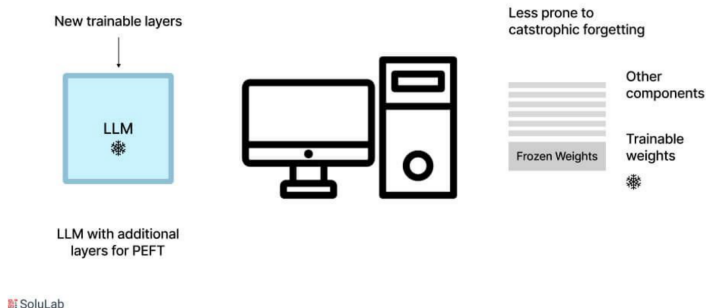
- ▶ **Definition:** Fine-tune all parameters of the pre-trained model on downstream data.
- ▶ **Historical Use:** Common in early GPT-2 and BERT applications.
- ▶ **Pros:**
 - Maximum flexibility and performance
- ▶ **Cons:**
 - Expensive (requires large compute resources)
 - Prone to catastrophic forgetting
 - Not efficient for large models

Full Fine-Tuning Workflow



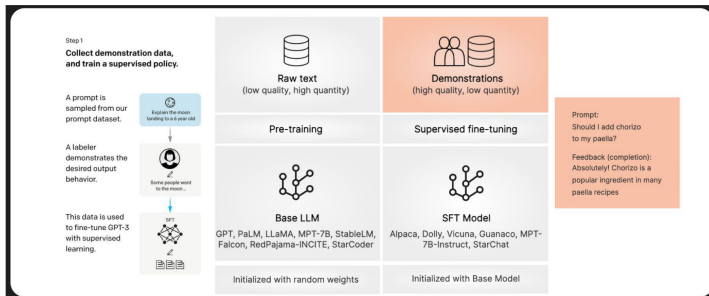
- ▶ **Key Idea:** Keep the base model frozen, tune only a small subset of parameters.
- ▶ **Types:**
 - Adapter Modules (Houlsby et al., 2019)
 - Prompt Tuning / Prefix Tuning
 - LoRA / QLoRA
- ▶ **Benefits:**
 - Enables multi-tasking and personalization
 - Suitable for low-resource adaptation
 - Reduces compute and memory requirements

Parameter efficient fine-tuning (PEFT)

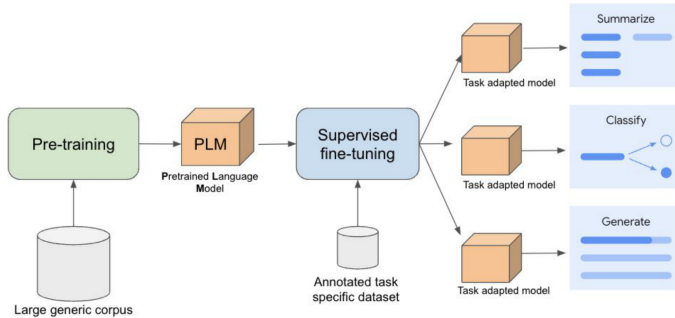


Supervised Fine-Tuning (SFT)

What is Supervised Fine-Tuning (SFT)?



Supervised Fine-Tuning For Gemini LLM



Supervised Fine Tuning for Gemini LLM

► Instructional datasets:

- **OpenAI:** InstructGPT (labeler demonstration + preference data)
- **Stanford Alpaca:** 52k GPT-3 generated instructions
- **Others:** Dolly, ShareGPT, OASST, UltraChat

► **Note:** Quality of supervision greatly affects model behavior.

- ▶ Cannot capture nuanced human preferences or values.
- ▶ May reinforce existing biases or hallucinations present in the data.
- ▶ Risk of overfitting, especially on synthetic or noisy datasets.
- ▶ Often leads to safe but bland and generic responses.

RLHF — Reinforcement Learning with Human Feedback

- ▶ SFT doesn't teach the model what humans prefer.
- ▶ RLHF introduces reward learning and policy optimization.
- ▶ Inspired by InstructGPT (Ouyang et al., 2022).

1. **Supervised Fine-Tuning (SFT):** Train the base model on instruction-response pairs.
2. **Reward Model (RM) Training:** Learn a reward function from human preference rankings.
3. **RL Optimization (e.g., PPO):** Optimize the language model to maximize the learned reward.

Step 1

**Collect demonstration data,
and train a supervised policy.**

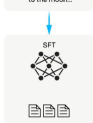
A prompt is
sampled from our
prompt dataset.

Explain the moon
landing to a 6 year old

A labeler
demonstrates the
desired output
behavior.

Some people want
to the moon...

This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.

Explain the moon
landing to a 6 year old

A B
Dislike gravity... Explain what...
C D
Moon is natural satellite of... People went to the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.



Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.

Write a story
about frogs

The policy
generates
an output.



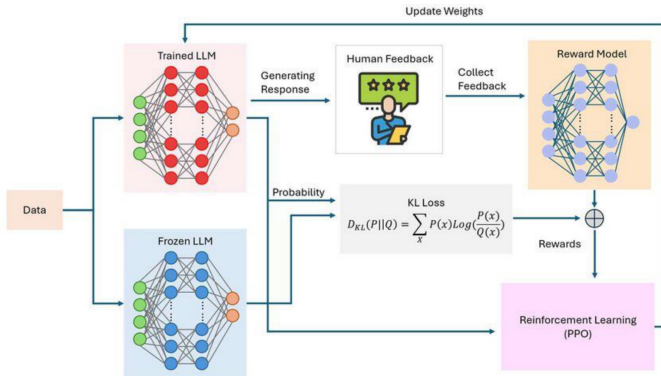
The reward model
calculates a
reward for
the output.

Once upon a time...



The reward is
used to update
the policy
using PPO.

r_k



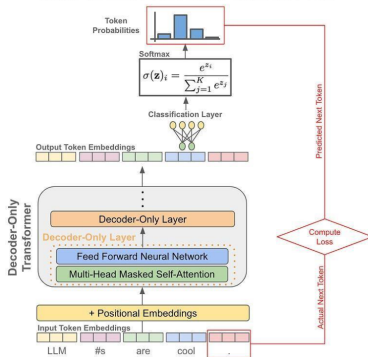
- ▶ **Given:** Two responses y_A and y_B to the same prompt.
- ▶ **Human Feedback:** Annotator selects which response is preferred.
- ▶ **Learning:** The reward model r_ϕ is trained to assign higher scores to preferred responses.
- ▶ **Pairwise Loss:**

$$L_{\text{RM}} = -\log \sigma \left(r_\phi(y_A) - r_\phi(y_B) \right)$$

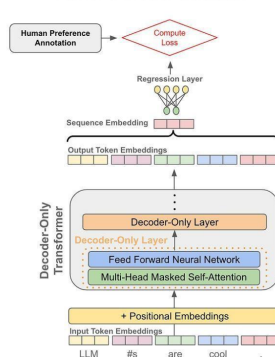
where σ is the sigmoid function.

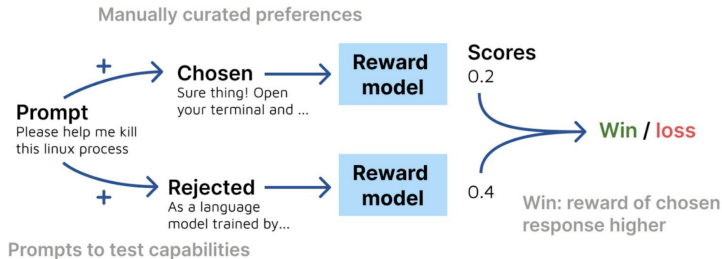
- ▶ The RM acts as a proxy for human judgment in downstream RL optimization.

Next-Token Prediction with an LLM



Reward Model Structure





- ▶ **Objective:** Maximize the reward given by the reward model (RM).
- ▶ **Algorithm:** Proximal Policy Optimization (PPO) is used to update the language model.
- ▶ **PPO Loss:**

$$L_{\text{PPO}} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ is the probability ratio, and $\hat{A}_t$ is the advantage estimate.

- ▶ **KL Penalty:** A KL-divergence penalty is added to keep the policy close to the base (SFT) model:

$$L_{\text{KL}} = \beta \text{KL}(\pi_\theta \parallel \pi_{\text{SFT}})$$

RLHF Training: Proximal Policy Optimization (PPO)

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



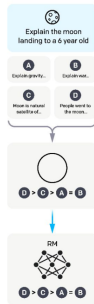
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

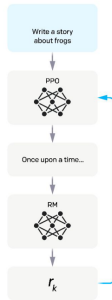
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



- ✓ Stable optimization
- ✓ Encourages exploration
- × Expensive (needs many rollouts)
- × Sensitive to reward model errors

- ▶ **New method:** Bypasses RL and reward model training entirely.
- ▶ **Key Idea:** Optimize the policy directly from human preference data.
- ▶ **Reference:** Rafailov et al., 2023.

DPO Loss:

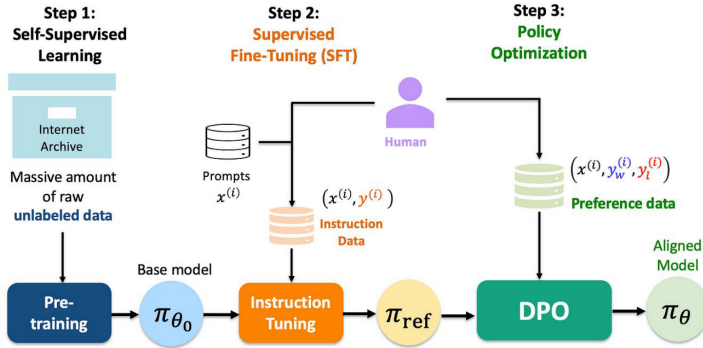
$$L_{\text{DPO}} = -\log \frac{\exp\left(\beta \log \frac{\pi(y^+)}{\pi_0(y^+)}\right)}{\exp\left(\beta \log \frac{\pi(y^+)}{\pi_0(y^+)}\right) + \exp\left(\beta \log \frac{\pi(y^-)}{\pi_0(y^-)}\right)}$$

where y^+ is the preferred response, y^- is the dispreferred response, π is the current policy, π_0 is the reference (SFT) policy, and β is a temperature parameter.

Benefits:

- ▶ Simpler and more stable than PPO
- ▶ No need for reward model training or RL rollouts

Direct Preference Optimization (DPO)



LoRA and Quantized LoRA

- ▶ **Key Idea:** Update low-rank matrices instead of full weights.
- ▶ For a weight matrix $W \in \mathbb{R}^{d \times k}$:

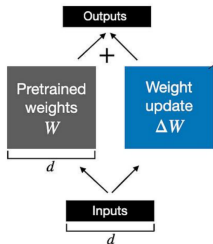
$$W' = W + \Delta W, \quad \Delta W = AB^T$$

where $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{k \times r}$.

- ▶ Only train $A, B \rightarrow$ drastically reduces parameters.

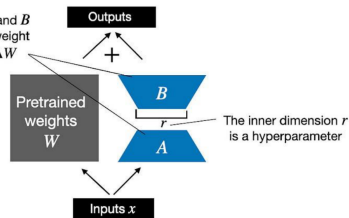
What is LoRA?

Weight update in **regular finetuning**



Weight update in **LoRA**

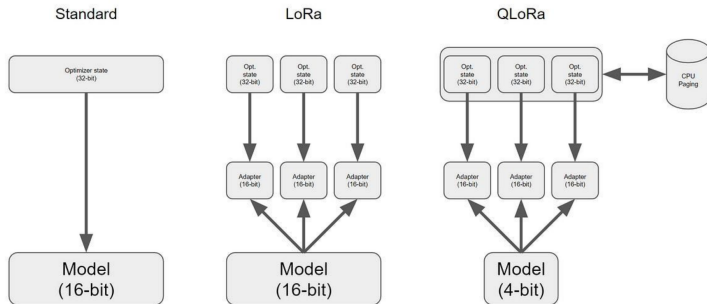
LoRA matrices A and B approximate the weight update matrix ΔW



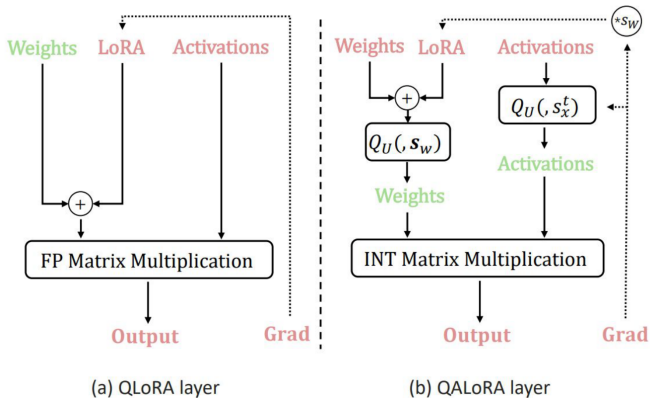
- ▶ **Very lightweight:** Only 0.1%–1% of parameters are trainable.
- ▶ **Hardware friendly:** Enables fine-tuning on consumer GPUs and even laptops.
- ▶ **Modular:** Supports plug-and-play adapters for different tasks or users.
- ▶ **Personalization:** Allows efficient user- or domain-specific adaptation.

- ▶ **Key Idea:** Fine-tune large language models in 4-bit precision using LoRA adapters.
- ▶ **Scalability:** Enables fine-tuning of 65B parameter models on a single 48GB GPU.
- ▶ **Efficiency:** Highly memory-efficient with no significant performance loss (Dettmers et al., 2023).
- ▶ **Techniques:**
 - Double quantization
 - Paged optimizers
 - NF4 (NormalFloat 4-bit) quantization format

QLoRA — Quantized LoRA



QLoRA — Quantized LoRA



▶ Instruction tuning for resource-constrained settings:

- Enables fine-tuning large models on modest hardware (e.g., consumer GPUs, laptops).
- Used for domain adaptation, personalization, and rapid prototyping.

▶ Popular fine-tuned models:

- **Alpaca, Guanaco, Vicuna, Mistral:** All leverage LoRA/QLoRA for efficient instruction tuning.

▶ Model merging and compositionality:

- Merge multiple LoRA adapters for multi-domain or multi-task capabilities.
- Compose adapters for new tasks without retraining the base model.

- ▶ **Data Quality:** RLHF relies heavily on high-quality, diverse preference data or human feedback; poor data leads to suboptimal or biased models.
- ▶ **Scalability:** Collecting human feedback at scale is expensive and time-consuming.
- ▶ **Alignment Gaps:** RLHF may not perfectly capture complex, nuanced human values or intent.
- ▶ **Overoptimization:** Models may "game" the reward model, leading to reward hacking or unnatural outputs.
- ▶ **Generalization:** Fine-tuning and RLHF can cause models to overfit to specific tasks or feedback distributions, reducing robustness.
- ▶ **Catastrophic Forgetting:** Risk of losing previously learned capabilities during aggressive fine-tuning.
- ▶ **Computational Cost:** Both fine-tuning and RLHF are resource-intensive, especially for large models.

- ▶ **Ethical Concerns:** Human preferences used for training may encode societal biases, under-representation, or other ethical issues.
- ▶ **Transparency:** RLHF pipelines can be hard to interpret and debug.

- ▶ **RLAIF (RL with AI Feedback):** Automate human labelers using strong AI models for preference data.
- ▶ **Constitutional AI:** Use rule-based alignment and self-critique to guide model behavior (e.g., Anthropic's approach).
- ▶ **Open RLHF Datasets:** Promote better community sharing and reproducibility with open preference datasets.
- ▶ **Multi-modal Alignment:** Extend alignment techniques to vision, audio, and code generation tasks.
- ▶ **Reward Hacking Defense:** Develop robust reward functions to prevent models from exploiting reward loopholes.

- ▶ Fine-tuning enables specialization and alignment of LLMs.
- ▶ Supervised Fine-Tuning (SFT) is the foundational step.
- ▶ RLHF incorporates human preference alignment using PPO.
- ▶ DPO streamlines the process with a direct preference loss.
- ▶ LoRA and QLoRA make adaptation efficient and accessible.
- ▶ The field is rapidly evolving with ongoing innovation.

1. Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. InstructGPT. <https://arxiv.org/abs/2203.02155>
2. Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. <https://arxiv.org/abs/2305.18290>
3. Dettmers, T., Pagnoni, A., Holtzman, A., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. <https://arxiv.org/abs/2305.14314>
4. Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with RLHF. <https://arxiv.org/abs/2204.05862>
5. Zhang, S., Roller, S., Goyal, N., et al. (2022). OPT: Open Pre-trained Transformer Language Models. <https://arxiv.org/abs/2205.01068>
6. Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. <https://arxiv.org/abs/2106.09685>
7. Anthropic (2022). Constitutional AI: Harmlessness from AI Feedback. <https://www.anthropic.com/>
8. HuggingFace. RLHF Course. <https://huggingface.co/learn/rlhf-course>

9. Stanford CRFM. *CRFM Slides*. <http://crfm.stanford.edu/>